

## **Optimization of Complex Decisions**

Project: Optimization of the maintenance planning of a fleet of  
offshore wind turbines

Patricia Call Cerezo

Adnan Khan

Students, Operation Research & Risk Analytics (ORRA)

## Index

|                                                                                        |   |
|----------------------------------------------------------------------------------------|---|
| 1. Introduction .....                                                                  | 3 |
| 2. Problem description .....                                                           | 3 |
| 3. Optimization Model (Q1) .....                                                       | 4 |
| 3.1 Mathematical Programming Model .....                                               | 5 |
| 4. Q2: Solving the model for the three instances                                       |   |
| 4. 1 The initial code                                                                  |   |
| 5. Q3: Logistical and economic optimization: The improved model (Sensitivity Analysis) |   |

## 1. Introduction

The rapid expansion of offshore wind power has made maintenance planning a critical operational challenge. Due to harsh marine conditions and mechanical wear, wind turbines progressively degrade, requiring effective maintenance strategies to minimize downtime and costs.

This project aims to develop an optimized maintenance strategy for a fleet of offshore wind turbines by modelling the problem as a structured mathematical optimization model. The goal is to determine when and how to perform maintenance operations (preventive or corrective) while considering limited crew resources, transportation constraints, and associated costs.

Beyond solving this specific problem, the project serves as a practical exercise in applying optimization techniques to real-world decision-making scenarios.

## 2. Problem description

Offshore wind turbines are complex industrial systems subject to gradual degradation caused by mechanical fatigue, wear, and exposure to harsh marine conditions. As turbines operate to generate electricity, their components deteriorate over time until reaching a critical failure threshold. Once a failure occurs, the turbine stops producing power, resulting in production downtime and revenue losses for the operator.

To mitigate this, two types of maintenance strategies are considered:

- Preventive Maintenance (**PM**): performed before a failure, within a predefined time window. It allows the operator to replace or inspect components to prevent failures. PM operations are generally cheaper but, if carried out too early, may waste useful component life.
- Corrective Maintenance (**CM**): performed after a failure occurs. It restores the turbine to an operational state but is more expensive and leads to production losses during the repair period.

The operator manages a fleet composed of several offshore wind farms, each containing a certain number of identical turbines. The planning horizon is discretized into weekly periods. Each turbine has an initial operating time and a maximum lifetime  $H$ , representing the number of weeks before failure if no maintenance is performed. A PM can only be scheduled when the turbine's operating time  $a$  satisfies the condition where  $H - h \leq a < H$ ,  $h$  defines the preventive maintenance window.

Maintenance operations require specialized maintenance crews, each equipped with vessels and technical resources. Each crew can:

- Work on at most  $k$  turbines per week.
- Operate at only one wind farm per week.

- If transferred between two farms, spend one week traveling during which no maintenance can be performed.

Every transfer between wind farms incurs a transportation cost  $c^{trans}$ . In addition, a turbine that is not operational causes an opportunity cost  $c^{op}$ , representing the revenue lost due to production downtime.

The operator must decide:

1. When to perform maintenance on each turbine.
2. Which type of maintenance (PM or CM) to apply.
3. How to allocate maintenance crews among the wind farms each week.

To minimize the total operational cost, which includes:

- The cost of PM and CM interventions.
- The opportunity cost of turbines not producing.
- The transportation cost of maintenance crews.

The objective is therefore to produce an optimal maintenance plan that ensures reliable turbine operation at minimum total cost, while respecting technical, temporal, and logistical constraints.

### 3. Optimization Model (Q1)

To design an efficient maintenance plan for the offshore wind turbine fleet, we formulated the problem as a Mixed-Integer Linear Programming (MILP) model with the objective of minimizing total operational costs.

We first developed the mathematical model by hand, defining decision variables, constraint equations, and the objective function. This exercise proved valuable for understanding the problem structure but revealed significant challenges:

- The large number of variables needed to track crew assignments and turbine states across all time steps.
- Complex interdependencies between constraints governing crew movements, turbine operating times, and maintenance decisions.
- Difficulty in manually verifying model completeness and consistency.

These observations motivated us to implement the model systematically using optimization software.

### 3.1 Mathematical Programming Model

We began by carefully identifying and defining all the sets needed to structure the problem. This initial step was crucial to establish a clear notation system and understand the problem's dimensions. Once the sets were properly defined, we proceeded to identify the parameters (input data) and decision variables.

#### Sets:

$i \in I_j = \{1, \dots, n_j\}$  turbines in windfarm  $j$

$j \in J = \{1, \dots, N\}$  windfarms

$r \in R = \{1, \dots, R\}$  crews

$t \in T = \{1, \dots, T\}$  time (weeks)

#### Parameters:

$a_{ij}$ : operating time of turbine  $i$  in windfarm  $j$

$H_i$ : maximum operating time before failure for turbine  $i$

$h_i$ : preventing time window for turbine  $i$

$k_{rt}$ : maximum number of interventions of crew  $r$  in week  $t$ .

$c_{ijt}^{CM}$ : CM cost for turbine  $i$  in windfarm  $j$  in week  $t$ .

$c_{ijt}^{PM}$ : PM cost for turbine  $i$  in windfarm  $j$  in week  $t$ .

$c_{rtj}^{trans}$ : Transportation cost of moving crew  $r$  to windfarm  $j$  in week  $t$ .

$c_{ijt}^{op}$ : Opportunity cost for turbine  $i$  in windfarm  $j$  in week  $t$ .

#### Variables:

$x_{ijt}^{PM} \in \{0,1\}$ : 1 if PM is done to  $(i,j)$  in week  $t$ .

$x_{ijt}^{CM} \in \{0,1\}$ : 1 if CM is done to  $(i,j)$  in week  $t$ .

$f_{ijt} \in \{0,1\}$ : 1 if turbine  $i$  in windfarm  $j$  is in failure in week  $t$ .

$w_{rjt} \in \{0,1\}$ : 1 if crew  $r$  works in windfarm  $j$  in week  $t$ .

$a_{ijt} \in \mathbb{R}$ : operating time (age) of turbine  $(i,j)$  at the start of week  $t$ .

$z_{rjt} \in \{0,1\}$ : 1 if crew  $r$  travels to windfarm  $j$  in week  $t$ .

$$\text{OBJ: } [MIN] \sum_{t \in T} \left( \sum_{i \in I} \sum_{j \in J} (c_{ijt}^{CM} \cdot x_{ijt}^{CM} + c_{ijt}^{PM} \cdot x_{ijt}^{PM} + c_{ijt}^{op} \cdot f_{ijt}) + \sum_{r \in R} (c_{rtj}^{trans} \cdot z_{rjt}) \right)$$

#### Constraints:

$$\text{One intervention per turbine: } x_{ijt}^{PM} + x_{ijt}^{CM} \leq 1 \quad \forall i \in I_j, j \in J, t \in T$$

$$\text{Initial operating time: } a_{ij1} = a_{ij} \quad \forall i \in I_j, j \in J$$

$$\text{Operating time (age) evolution: } a_{ij,t+1} = a_{ijt} + 1 - (a_{ijt} + 1) \cdot (x_{ijt}^{PM} + x_{ijt}^{CM}) \quad \forall i \in I_j, j \in J, t < T$$

$$\text{Operating time window: } a_{ijt} \geq (H_i - h_i) \cdot x_{ijt}^{PM}; a_{ijt} \leq (H_i - 1) \cdot M(1 - x_{ijt}^{PM})$$

Failure:  $a_{ijt} - H < M \cdot (1 - f_{ijt}) \quad \forall i \in I, j \in J, t \in T; f_{ijt} \leq \frac{a_{ijt}}{H} \quad \forall i \in I, j \in J, t \in T$

If a turbine is in failure, it stays in failure unless it's repaired:  $f_{ij,t+1} \geq f_{ijt} - x_{ijt}^{CM} \quad \forall i \in I, j \in J, t < T$

Can't do PM on a failed turbine:  $x_{ijt}^{PM} \leq 1 - f_{ijt} \quad \forall i \in I, j \in J, t \in T$

Only CM on failed turbines:  $x_{ijt}^{CM} \leq f_{ijt} \quad \forall i \in I, j \in J, t \in T$

PM window:  $(H - h) \cdot x_{ijt}^{PM} \leq a_{ijt} \quad \forall i \in I, j \in J, t \in T$

Maximum  $k$  turbines per crew per week:  $\sum_{i \in I_j} (x_{ijt}^{PM} + x_{ijt}^{CM}) \leq \sum_{r \in R} k \cdot w_{rjt} \quad \forall j \in J, t \in T$

One crew per windfarm each week:  $\sum_{j \in J} w_{rjt} \leq 1 \quad \forall r \in R, t \in T$

We can only do maintenance if there is a crew in the windfarm:  $x_{ijt}^{PM} + x_{ijt}^{CM} \leq \sum_{r \in R} w_{rjt} \quad \forall i \in I, j \in J, t \in T$

Only one travel per week per crew:  $\sum_{j \in J} z_{rjt} \leq 1 \quad \forall r \in R, t \in T$

If a crew is travelling, it can't work:  $w_{rjt} + z_{rjt} \leq 1 \quad \forall r \in R, j \in J, t \in T$

Having established a comprehensive Mixed-Integer Linear Programming (MILP) model by hand, detailing the objective function, decision variables, and constraints, the next crucial step was to transition this mathematical formulation into a computationally solvable format. Although the manual derivation was essential for deeply understanding the problem's complex structure, which involves tracking turbine age, crew movements, and maintenance timing, implementing it via a dedicated optimization library, such as Gurobi, was necessary to handle the large number of variables and constraints across the fleet and time horizon. This programming phase allows for the rapid testing of different fleet instances and numerical parameters, providing the concrete solutions and performance metrics required to complete the project's analysis tasks.

## 4. Q2: Solving the model for the three instances

In this section, we solve the optimization model developed in Q1 for the three provided instances of the offshore wind turbine maintenance planning problem. The goal is to obtain a cost-minimizing maintenance schedule that satisfies all operational and resource constraints.

Initially, the Gurobi solver was tested for this task. However, due to license limitations and the large size of the problem instances, the solver was unable to handle all cases efficiently. As a result, we switched to the open-source HiGHS solver, which supports large-scale linear and mixed-integer optimization problems and integrates seamlessly with Pyomo.

### 4. 1 The initial code

Our initial code is made of 6 different cells.

#### Cell 1: Imports and solver configuration

This cell initializes the computational environment. It imports the necessary libraries and sets up the mechanism for solving the optimization model.

```
from __future__ import annotations
from pathlib import Path
import re
from typing import Dict, List, Tuple

from pyomo.environ import (
    ConcreteModel, Set, Param, Var, NonNegativeReals, NonNegativeIntegers, PositiveIntegers,
    Binary, Objective, Constraint, minimize, SolverFactory, value
)

def choose_solver():
    """Attempts to find an available MILP solver in your environment."""
    for cand in ["gurobi", "highs", "glpk", "cbc"]:
        try:
            s = SolverFactory(cand)
            if s.available(False):
                return cand
        except Exception:
            pass
    raise RuntimeError(
        "No solver found. Please install one: 'pip install highspy' (HiGHS) "
        "or use Gurobi/CPLEX/GLPK."
    )
```

This block imports `pyomo.environ`, the core library used to formulate the MILP model. Other utilities like `pathlib` and `re` are included for file operations. The `choose_solver()` function automatically detects and selects an available solver on the system (e.g., HiGHS, GLPK, Gurobi).

Cell 2 & 3: Instance data parsing

```

from __future__ import annotations
from collections import defaultdict
import re
from typing import Dict, List, Tuple

def parse_instance_txt(file_path: str) -> Dict:

    instance = {}

    # Regular expression to capture variables like 'k = 4'
    pattern = re.compile(r'^\s*([a-zA-Z_]+\s*=\s*(\d+))')

    # -----
    # 1. Reading header and scalar parameters
    # -----
    try:
        with open(file_path, 'r') as f:
            lines = f.readlines()
    except FileNotFoundError:
        print(f"Error: File not found at path {file_path}")
        return {}

    age_lines_start = -1

    for i, line in enumerate(lines):
        line = line.strip()
        if not line or line.startswith('#'):
            continue

def parse_instance_txt(path: str | Path):
    """
    Reads your P1 instance .txt files and returns a dict with all parameters.
    Supports 'NA' inside the initial age matrix.
    """
    path = Path(path)
    text = path.read_text(encoding="utf-8")

    # Extracts scalars k, H, h, c_PM, c_CM, c_OP, c_transport, N, R, n_max
    def get_scalar(name):
        m = re.search(rf'^\s*{name}\s*=\s*([0-9]+\s*$)', text, re.M)
        if not m:
            raise ValueError(f"'{name}' not found in {path.name}")
        return int(m.group(1))

    k = get_scalar("k")
    H = get_scalar("H")
    h = get_scalar("h")
    c_PM = get_scalar("c_PM")
    c_CM = get_scalar("c_CM")
    c_OP = get_scalar("c_OP")
    c_transport = get_scalar("c_transport")
    N = get_scalar("N")
    R = get_scalar("R")
    n_max = get_scalar("n_max")

    # Reads the N rows of the a0 matrix (may contain NA)
    # We take the last N lines with commas (robust to comments).
    lines = [ln.strip() for ln in text.splitlines() if "," in ln]
    if len(lines) < N:
        raise ValueError("Not enough rows with a0 data in the file.")

```

These cells handle the crucial task of reading and structuring the raw data from the .txt instance files into Python parameters usable by the Pyomo model. The `parse_instance_txt()` function reads scalar parameters from the file header using regular expressions. It then processes the initial age matrix, handling NA values to correctly identify the set of existing turbines and their initial operating ages. Cell 3 includes a test block to quickly confirm that the files (Small, Medium, Large) are correctly loaded, ensuring all parameters and age data are properly mapped before the model construction begins.

#### Cell 4: MILP model construction

This cell translates the mathematical formulation of the maintenance planning problem (Q1) into a functional optimization model using Pyomo's syntax.



```

# =====
# Cell 4 (ORIGINAL LOGIC) - Model Construction
# =====

from pyomo.environ import ConcreteModel, Set, Param, Var, NonNegativeReals, NonNegativeIntegers, PositiveIntegers, Binary, Objective, Constraint, minimize

def build_model(instance, T_max=None):
    """
    Constructs the P-1 MILP based on the parsed instance.
    """
    k = instance["k"]
    H = instance["H"]
    h = instance["h"]
    c_PM = instance["c_PM"]
    c_CM = instance["c_CM"]
    c_OP = instance["c_OP"]
    c_transport = instance["c_transport"]
    N = instance["N"]
    R = instance["R"]
    farms_I = instance["farms_I"] # dict: j -> [i1,i2,...]
    a0 = instance["a0"] # dict: (j,i) -> initial age

    # Default horizon
    if T_max is None:
        T_max = H

    m = ConcreteModel()

    # --- Sets ---
    m.J = Set(initialize=range(1, N+1))
    m.R = Set(initialize=range(1, R+1))
    m.T = Set(initialize=range(1, T_max+1), ordered=True)
    m.Ij = Set(m.J, initialize=lambda m,j: farms_I[j])

```

The `build_model` function defines the problem's components such as sets, variables, objective function and all the constraints already stated in Q1.

## Cell 5: solve and results summary

This function executes the optimization and presents the results in a structured and readable format.

```

# =====
# Cell 5 - Solve and display results
# =====

from pyomo.environ import SolverFactory, value
import pandas as pd

def solve_and_summarize(instance, T_max=None, solver_name=None, tee=False):
    # Build the model
    m = build_model(instance, T_max=T_max)

    # Choose solver
    if solver_name is None:
        try:
            solver_name = choose_solver()
        except RuntimeError:
            print("Automatic solver not found, using 'glpk'.")
            solver_name = "glpk"

    solver = SolverFactory(solver_name)

    # Solve
    print(f"Starting solver '{solver_name}'...")
    res = solver.solve(m, tee=tee)

    try:
        obj = float(value(m.OBJ))
        print(f"\n--- Solution Summary (T_max={m.T.last()}) ---")
        print(f"Solver: {solver_name}, Status: {res.solver.termination_condition}")
        print(f"OBJ total = {obj:.2f}")

    # --- Cost Breakdown ---

```

This block handles the solution process. After selecting the solver (via `SolverFactory`), it executes the `solve(m)` command to find the optimal plan. Post-solution, it calculates the Cost Breakdown (PM, CM, OP, TR) to analyze cost drivers. Finally, it extracts the binary variables and organizes them into readable tables (Pandas DataFrames) detailing the Actions (PM/CM)

and Crew Work/Travel Schedule over the planning horizon, simplifying the analysis of the optimal policy.

## Cell 6: final execution loop

This cell manages the execution of the model across all three instances (Small, Medium, Large).

```
from collections import defaultdict

# --- INSTANCE CONFIGURATION ---
paths = {
    "small": "Maintenance_planning-instance_small_P1.txt",
    "medium": "instance_medium_P1.txt",
    "large": "instance_large_P1.txt"
}

# Horizon Limits (T_max) to reduce computation time
T_LIMITS = {
    "small": 30,
    "medium": 30,
    "large": 30,
}

# -----

models = {}
objs = {}
for name, p in paths.items():
    print(f"***{name}***")
    print(f"Solving instance: {name} -> {p}")

    current_t_max = T_LIMITS.get(name)
    print(f"*** Using T_max = {current_t_max} ***")

    # 1. Parse the instance (The parser function already applies the cost hack: c_transport = 5)
    inst = parse_instance_txt(p)

    # 2. Solve with HiGHS and T_max
    m, obj = solve_and_summarize(inst, T_max=current_t_max, solver_name="highs", tee=False)
```

After executing this code, we got the following results:

```
Solving instance: small -> Maintenance_planning-instance_small_P1.txt
*** Using T_max = 30 ***
Starting solver 'highs'...
```

```
--- Solution Summary (T_max=30) ---
Solver: highs, Status: optimal
OBJ total = 230.00
```

```
--- Cost Breakdown ---
Cost PM: 0.00
Cost CM: 0.00
Cost OP: 230.00 (Failure)
Cost TR: 0.00 (Transport)
TOTAL: 230.00
```

```
--- Action Planning (PM/CM/Work) ---
No maintenance or work action was scheduled.
```

```
--- Travel Planning (Transport) ---
No travel was scheduled.
```

```

Solving instance: medium -> instance_medium_P1.txt
*** Using T_max = 30 ***
Starting solver 'highs'...

--- Solution Summary (T_max=30) ---
Solver: highs, Status: optimal
OBJ total = 2720.00

--- Cost Breakdown ---
Cost PM: 0.00
Cost CM: 0.00
Cost OP: 2720.00 (Failure)
Cost TR: 0.00 (Transport)
TOTAL: 2720.00

--- Action Planning (PM/CM/Work) ---
No maintenance or work action was scheduled.

--- Travel Planning (Transport) ---
No travel was scheduled.

```

The optimization model (MILP) consistently found an optimal solution for all three instances by selecting the null action: do absolutely nothing.

| Instance | Optimal Total Cost | Cost PM | Cost CM | Cost TR | Cost OP (Failure) |
|----------|--------------------|---------|---------|---------|-------------------|
| Small    | 230.00€            | 0.00€   | 0.00€   | 0.00€   | 230.00€           |
| Medium   | 2,720.00€          | 0.00€   | 0.00€   | 0.00€   | 2,720.00€         |
| Large    | 8,736.00€          | 0.00€   | 0.00€   | 0.00€   | 8,736.00€         |

The model's optimal plan for the 30-week horizon is to schedule no maintenance and no travel. This is the most extreme form of the "Wait-to-Fail" strategy. The total cost is composed entirely of the Operational Loss accumulated from all turbines that are already failed or will fail within the planning horizon

The zero-action solution is a direct consequence of the overwhelming cost imbalance in the original parameters:

- Prohibitive transport cost: The cost to initiate any action is the transport cost ( $c^{trans} = 50\text{€}$  for Small/Large, 40 € for Medium).
- Low penalty for failure: The penalty for a turbine being failed ( $c^{OP} = 2$  per week) is too low to justify the crew's operational cost. for the small instance ( $c^{trans} = 50\text{€}$ ,  $c^{CM} = 25\text{€}$ , the cost of a single corrective action plus transport is 75€. To justify this expenditure, the turbine must be failed for  $75 / 2 = 37.5$  weeks.
- Horizon limit: Since the planning horizon is only  $T=30$  weeks, the model cannot recoup the high fixed cost of transport and repair through future revenue protection. The mathematically optimal decision is to set all decision variables to zero to minimize all variable costs, accepting the unavoidable fixed cost of lost production.

The results clearly demonstrate that the "do nothing" strategy is economically catastrophic as the fleet size increases:

| Instance | Total Cost | % Increase (vs Small) |
|----------|------------|-----------------------|
| Small    | 230.00€    | Base                  |
| Medium   | 2,720.00€  | $\approx 1,083\%$     |
| Large    | 8,736.00€  | $\approx 3,698$       |

The cost increases dramatically because the larger instances contain more turbines closer to the failure threshold ( $H=30$ ), leading to a massive increase in the cumulative Operational Loss when no action is taken.

In conclusion, this baseline model establishes that the maintenance problem, as originally defined by the parameters, is ill-posed for preventative action. The current cost structure dictates that inaction is the optimal decision to minimize variable costs, even though this leads to an unacceptable and escalating loss of revenue from failures. This baseline provides the necessary evidence to justify the parameter modifications applied in the next phase (Q3).